AI Hardware Hacking: What It Is and How to Secure TCIMO & TRITONE

Nov 4, 2026 | @m1 (Enhanced Edition)[1]

Bottom Line (30-Second Read)

"AI hardware hacking" is not new physics. AI just makes six known hardware attacks cheap enough that non-experts can run them overnight[1]. Your best defense is to fix the physical root causes and run the same AI and formal tools on your own designs before tape-out[1].

- **What Altman said (24 Oct 2026):** Models are finding bugs "outside the normal threat model... hardware-level, insane stuff," and "I no longer think we're gonna be able to just patch all the bugs." He gave zero public hardware examples[1].
- **What Amodei said (May 2026):** Anthropic's *Mythos Preview* found tens of thousands of mostly software vulnerabilities, giving defenders a 6-to-12-month window to patch before rival models catch up[1].
- **What is actually documented:** *Mythos Preview* chained exploits in OS kernels, browsers, FreeBSD's NFS server, and OpenBSD[1]. That is **software with source code**, not physical chips (and it failed to exploit several kernel bugs it flagged)[1]. It plausibly extends to Verilog/SystemVerilog RTL, which is also source code, but an RTL bug is not automatically an exploitable physical chip[1].
- **What it means for you:** Attacks that used to cost too much specialist time are now an overnight compute job for an AI agent[1]. Design **TCIMO**, **TRITONE**, and your **Tang Nano 9K** prototype assuming someone will run them[1].

1. The Six Things AI Is Doing to Hardware (And How to Stop Them)

You do not need a step-by-step exploit script to defend your chip—you only need to break the **physical or logical root cause**[1]. If the root cause is gone, the attack fails no matter how smart the attacker's AI is[1].

Family	What AI Does Now	Documented Evidence	What the Attacker Still Needs	Root Cause (Why It Works)	The Rule That Stops It
1. RTL & Firmware Bugs	LLMs read Verilog and flag CWE bugs	*BugWhisperer*, *LASHED*, *SecureRAG*	Your RTL source code (open-source RTL is	Unchecked state paths, open debug ports, and	**Formal proofs (SymbiYosys)** over all

	(debug unlocks, bad resets, clearable locks); formal tools filter false alarms[1].	*-RTL* (+42% detection across 18 LLMs), *AssertionLLM*[1].	exposed on day one)[1].	bad resets survive when nobody tests for them[1].	input sequences, not just directed tests[1].
2. Side-Channel Analysis (SCA)	Neural nets (CNNs/MLPs) learn power/EM-to-secret mappings directly from raw traces without hand-built leakage models[1].	Masure et al., Hoang et al. (TCHES 2020), *MaskedNet*, *SCARE*, *PowerGAN*[1].	Your chip on a bench + an oscilloscope or EM probe[1].	Switching transistor current depends on the data/weights it handles[1].	**Make current independent of secrets:** balanced cells, flattened ADC, masking. Verify with TVLA[1].
3. Fault Injection (Glitching)	RL agents and optimizers search voltage, clock, and EM glitch parameters unattended overnight[1].	ePrint 2026/536, Keysight AI tuner, Keysight NPU glitch agents, *CIM-FI* (2026)[1].	Physical access + a voltage/EM glitcher[1].	Driving a circuit outside voltage/timing specs latches bad values or skips security checks[1].	**Detect out-of-spec conditions & fail safe:** droop/glitch/EM sensors + instant reset on alarm[1].
4. Hardware Trojans	LLMs find low-visibility nets in RTL and write stealthy triggers/payloads that blend in[1].	NYU CSAW AI Hardware Attack Challenge (novices succeeded)[1].	Write access to your RTL repo or the AI assistant writing it[1].	Logic that rarely toggles is rarely reviewed, so hidden additions survive synthesis[1].	**100% coverage + independent formal properties;** delete dead logic; check netlist vs. RTL with

					eqy[1].
5. Accelerator Timing Leaks	Classifiers time power-gating wake-ups and data-dependent latency to steal MoE routing and training membership[1].	*GATEBLEED* (Intel Xeon AMX, MICRO 2025 / IEEE Micro Top Pick)[1].	Co-resident code on the same machine or 1-hop network (no physical access)[1].	Job duration or unit wake-up time depends on secret weights or sparsity[1].	**Fixed-latency execution** proven with a two-copy formal miter check[1].
6. Emerging Memory / CIM Physics	ML recovers ternary crossbar weights and inputs from noisy ADC power traces and chip impedance[1].	*DExiM* (commercial ReRAM, MICRO 2025), DATE 2026 (28 nm CIM), TU Delft CIM[1].	The chip on a bench[1].	Crossbar current is the sum of conductances, ADC power tracks output code, and PDN impedance reflects state[1].	**Constant-sum ternary cells + flattened ADC + isolated PDN** so current/impedance never track weights[1].

Quick Details on the Key Papers

- **SecureRAG-RTL & LASHED (Family 1):** Standard LLMs struggle with Verilog due to limited training data[13]. *SecureRAG-RTL* pairs multi-agent LLMs with retrieval-augmented hardware CWE templates to boost bug detection by 42% across 18 models[1]. *LASHED* combines LLMs with static taint/control-flow analysis[3].
- **GATEBLEED on Intel AMX (Family 5):** Intel Xeon AMX accelerators power-gate idle matrix tiles to save energy[6]. Waking a cold AMX unit takes ~20,000 cycles longer than a warm unit[6]. By timing this gap (even across VMs, SGX enclaves, or a 1-hop network), attackers leaked Mixture-of-Experts (MoE) routing with **100% accuracy**, ran hardware-only training data membership inference at **81% accuracy** (0.89 precision), built a covert channel **70,000x faster** than NetSpectre, and extracted 64-element weight sparsity in 50 minutes (**631% faster** than Hertzbleed)[6].

- **DExiM & Analog CIM Leaks (Family 6):** In ReRAM ($\mathrm{Ti/HfO_x/Pt}$ Valence Change Mechanism cells), bit-line current follows Kirchhoff's law: $I_i = \sum_{i=1}^{N} V_i G_{ii}$ [11]. Even when idle, theLow-Resistance (LRS) vs. High-Resistance (HRS) filament states change the chip's power delivery network (PDN) impedance[8]. *DExiM* (MICRO 2025) measured package impedance on commercial ReRAM to extract stored data non-invasively[1].

2. Threat Map for Your Three Targets

- **TCIMO (Ternary ReRAM CIM Macro): Highest physical risk.** Its architecture (ternary weights in a ReRAM crossbar read out via column ADCs) matches Family 6 directly[1].
- **TRITONE (ofFBeaT9/Tritone-TPU-SoC): Highest RTL & timing risk.** It integrates a 27-trit dual-issue superscalar RISC CPU with a 64x64 ternary TPU in CMOS[17]. Large AI-written RTL + ternary zero-skipping expose it to Trojans (Family 4), lock/debug bugs (Family 1), and sparsity timing leaks (Family 5)[1].
- **Tang Nano 9K + Raspberry Pi Prototype:** Uses a Gowin GW1NR-9C FPGA connected to a Pi[1]. Main risks are public RTL, unencrypted bitstreams, and link fuzzing[1].

TCIMO Analog Physics and TRITONE AI-Generated RTL Concentrate the Highest Pre-Silicon Security Risks

EMPIRICAL ATTACK FEASIBILITY BENCHMARKS (LITERATURE BASELINES)

State-of-the-Art Machine Learning Hardware Exploits

RTL BUG DETECTION

+42%

Detection Gain

SecureRAG-RTL across 18 LLMs

COLD-START DELAY

20k cyc

AMX Latency Gap

GATEBLEED power-gating leak

MODEL CONFIDENTIALITY

100%

MoE Routing Leak

Zero-logit hardware timing attack

SPARSITY EXTRACTION

631%

Faster Weight Recovery

AMX gating vs. Hertzbleed

MULTI-TARGET ARCHITECTURAL THREAT SEVERITY MATRIX

High Medium Low / N/A

ATTACK FAMILY	TCIMO Ternary ReRAM CIM	TRITONE 27-Trit RISC + TPU SoC	Tang Nano 9K + Pi FPGA & Host Prototype
1. RTL Bug Finding Automated LLM fuzzing	LOW Hardened ReRAM analog macro with minimal digital wrapper logic.	HIGH AI-generated ternary RTL prone to unverified corner cases and state bugs.	LOW Constrained bitstream verified through standard FPGA simulation suites.
2. Side-Channel Analysis Power & EM radiation	MEDIUM Integrated bitline currents leak dot-product intermediate ternary weights.	HIGH 64×64 TPU PE array and 27-trit ALU current surges produce distinct traces.	MEDIUM Exposed PCB traces and SPI header pins vulnerable to EM probe interception.
3. Fault Injection Voltage glitching & pulses	HIGH Voltage pulsing induces conductance drift and read/write disturb.	MEDIUM Supply glitching corrupts ternary register latching and TPU sequencers.	MEDIUM Board VDD rail dipping induces FPGA state machine desynchronization.
4. Hardware Trojans Stealthy payload insertion	LOW Dense analog crossbar topology restricts stealthy circuitry modification.	HIGH Automated LLM RTL generation pipeline susceptible to prompt-injected triggers.	LOW Constrained 9K LUT fabric leaves negligible spare routing for Trojan circuits.
5. Timing Leaks Accelerator & bus latency	MEDIUM Asymmetric multi-level ReRAM write latencies reveal programming activity.	HIGH Ternary ALU cycle differences and TPU gating stalls permit model extraction.	HIGH Exposed Raspberry Pi SPI/GPIO bus delays reveal exact batch boundaries.
6. Memory/CIM Physics Analog non-idealities	HIGH ReRAM thermal drift, cycle variability, and sneak paths degrade ternary accuracy.	LOW Pure digital CMOS registers and TPU buffers; zero non-volatile analog drift.	N/A Synchronous SRAM FPGA architecture; devoid of analog in-memory physics.

Quantitative benchmarks from documented hardware attacks (top) illustrate the speed and accuracy of automated exploitation, while the architectural threat matrix (bottom) maps each attack family across TCIMO, TRITONE, and the Tang Nano 9K prototype. High-severity vectors (red) require mandatory pre-tape-out architectural mitigations and formal or SPICE-level verification gates.

Data sources: NC State ECE GATEBLEED, IEEE MICRO GATEBLEED, IEEE Computer Architecture Letters, GitHub ofFBeaT9/Tritone-TPU-SoC

Two Hidden Design Traps to Watch Out For

1. **TRITONE's Zero-Skipping Trap:** Skipping zero weights ($w = 0$) is the main speed advantage of balanced ternary ($\{-1, 0, +1\}$)[1]. However, if sparse weights finish faster, anyone timing the job learns your model's weight sparsity (just like *GATEBLEED*)[1]. **Fix:** Make fixed-latency execution the default and lock zero-skipping behind a privileged opt-in bit[1].
2. **TCIMO's Drift vs. Reprogramming Trap:** ReRAM conductances drift over time and need periodic reprogramming[1]. But if writing -1, 0, or $+1$ takes a different number of write pulses, reprogramming in the field leaks the weights on the power rail and wears out the cells[1]. **Fix:** Always run the exact same maximum pulse count (pad early finishes with non-switching dummy pulses) and guard rows with a CRC/MAC[1].

3. What the Pre-Silicon Leakage Test (tvla_ternary.py) Showed

We built tvla_ternary.py and tested a behavioral model of one **64-row ternary ReRAM column**[1].

It runs Welch's two-sample t-test comparing power traces with **fixed weights** against traces with **random weights**:

$$t = \frac{\mu_{\text{fixed}} - \mu_{\text{rand}}}{\sqrt{\frac{s^2_{\text{fixed}}}{N_{\text{traces}}} + \frac{s^2_{\text{rand}}}{N_{\text{traces}}}}}$$

- **The Rule:** Any $|t| > 4.5$ means the power draw statistically leaks the stored weights[1].
- **The Winner: Constant-sum cell encoding + a current-flattened ADC + 0.5% programming accuracy** was the only combination that passed at high trace counts[1].

Design Tested	Traces per Group	Programming Variation	Seeds	Array |t|	ADC |t|	Verdict (|t| < 4.5 Passes)
1. Plain unbalanced cells + code-dependent ADC	20k	3.0%	1	379.0	86.0	**Leaks badly** (Array & ADC fail)[1]

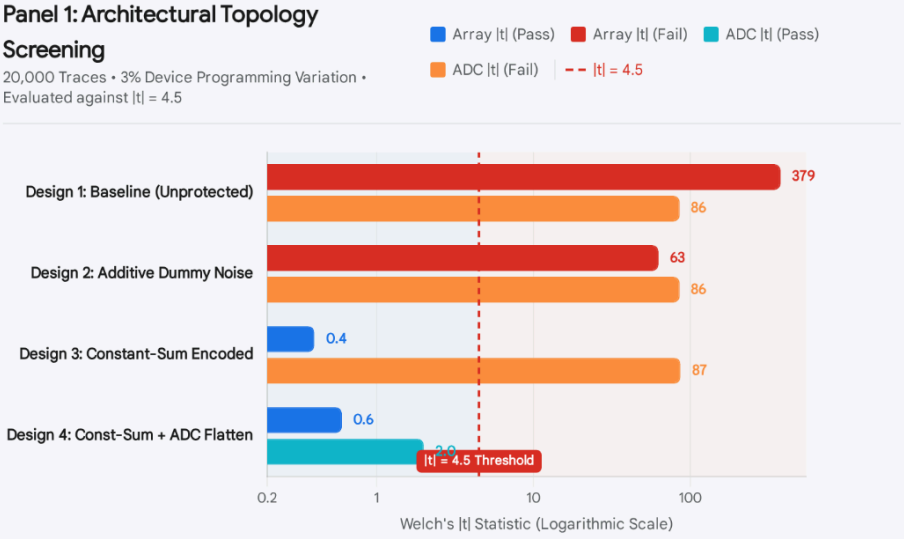
2. Design 1 + random dummy-cell noise	20k	3.0%	1	63.0	87.0	**Leaks** (Noise cuts array |t| 6x, still fails)[1]
3. Constant-sum cells (w=0 as 2x mid-conductance) + code-dependent ADC	20k	3.0%	1	0.4	87.0	**Leaks at ADC** (Array passes, ADC fails)[1]
4a. Constant-sum cells + ideal flat ADC	20k	3.0%	1	2.0	0.6*	**Pass** (at 20k traces)[1]
4b. Same (Design 4) at 7.5x more traces	150k	3.0%	5	1.2 - 6.4	0.6*	**Borderline / Fail** (1 of 5 seeds breaches 4.5)[1]
4c. Same (Design 4) with tight write-verify	150k	0.5%	5	0.2 - 1.1	0.6*	**Pass** (All seeds stay well below 4.5)[1]

*Note: The behavioral model sets the flat ADC's current to a constant, so it passes automatically; verify real leftover transistor imbalance in SPICE[1].

Constant-Sum Encoding, ADC Flattening, and **0.5% Write-Verify Accuracy** Are All Required to Pass TVLA

Panel 1: Architectural Topology Screening

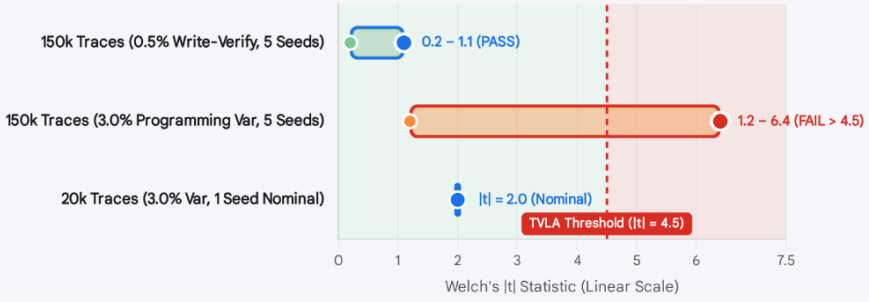
20,000 Traces • 3% Device Programming Variation • Evaluated against |t| = 4.5



Design 1	Design 2 (Dummy Noise)	Design 3 (Const-Sum)	Design 4 (Dual-Mitigated)
Array & ADC Fail	Array & ADC Fail	ADC Only Fails	100% TVLA PASS

Panel 2: Device Mismatch Stress Test on Design 4

150,000 Traces • Multi-Seed Mismatch Span • Frozen Cell Leakage Scaling



🔒 **Attacker Workload Barrier:** Tightening write-verify programming accuracy to **0.5%** compresses the worst-case leakage to **|t| = 1.1**. Because Welch's t-statistic scales with $\sqrt{N}$, this pushes the estimated trace requirement to breach |t| = 4.5 from 150k up to **~2,500,000 traces**, rendering practical side-channel key extraction infeasible.

Pre-silicon Welch's t-test screening on a 64-row ternary ReRAM column demonstrates that dummy noise alone only reduces array |t| from 379 to 63 (top). While constant-sum encoding eliminates the primary array leak (|t| = 0.4), the peripheral ADC continues to leak (|t| = 87) until flattened. At 150,000 traces (bottom), 3% programming variation causes frozen cell mismatch to breach the |t| = 4.5 threshold (reaching 6.4), whereas tightening write-verify tolerance to 0.5% suppresses worst-case |t| to 1.1.

Data sources: Tritone-TPU-SoC Repository (tvla_ternary.py screening)

4 Takeaways from the Test

1. **Noise raises cost but never fixes the leak:** Dummy cells dropped array $|t|$ from 379 to 63—still a massive fail[1]. An attacker using AI side-channel tools (Family 2) simply collects more traces[1]. Use ADC dither only as a backup[1].
2. **The ADC is a separate leak:** Balancing the ReRAM cells fixed the crossbar ($|t| = 0.4$), but the ADC kept leaking ($|t| = 87$) because its current tracked the output code[1]. You must flatten the ADC current separately[1].
3. **Device mismatch is a frozen leak:** Every programmed cell keeps a static programming error σ_{prog} [1]. Because $|t|$ grows with $\sqrt{N_{\text{traces}}}$, 3% variation passed at 20k traces ($|t| = 2.0$) but failed at 150k traces ($|t|$ up to 6.4)[1]. Tightening variation to **0.5%** kept the worst seed at **1.1**—forcing an attacker to collect **~2.5M traces** instead of 150k[1].
4. **0.5% is a hard target for ReRAM:** Hitting 0.5% on the mid-conductance state ($w = 0$) requires closed-loop write-verify stepping[1]. Also note: constant-sum cells only hide **weights**, not **inputs** (inputs need masking $x + r$ in Checklist Item 2)[1].
   - **Run on your own SPICE traces:** python tvla_ternary.py file fixed.npy random.npy[1]
   - **Sweep behavioral model:** python tvla_ternary.py demo <sigma> <traces> <seed>[1]

4. Priority Defence Checklist (Done Only When the Test Passes)

Every item below has a strict pass/fail test so a reviewer can verify it without taking anyone's word[1]. Before tape-out, also run a **deep-learning leakage attack (MLP/CNN)** on your SPICE traces in addition to TVLA[1].

#	Target	Do This	Pass When (Verification Gate)	Tools	Est. Cost
1	**TCIMO**	**Current-flattened ADC;** use voltage reference dither only as a supplement [1].	TVLA |t| < 4.5 (fixed vs. random weights) on extracted SPICE ADC supply current[1].	SPICE + tvla_ternary.py[1]	+10-30% ADC power[1]

2	**TCIMO**	**Balanced (constant-sum) weight encoding** + dummy columns + input masking ($x + r$) + shuffled row order[1].	1st- and 2nd-order TVLA pass at 10x the trace count an attacker can practically collect[1].	tvla_ternary.py[1]	2x cells, 2-5x latency[1]
3	**TCIMO**	**Tight write-verify (0.5% target);** periodic reprogram/remap; CRC/MAC on stored weights[1].	Mismatch TVLA passes at 150k+ traces; drift model proves CRC/MAC fires before accuracy drops[1].	Monte-Carlo SPICE[1]	Slower weight write[1]
4	**TCIMO**	**Constant-time programming:** always run the maximum write pulse count for every cell[1].	Pulse-count distribution is identical across trits -1, 0, and +1[1].	RTL Testbench[1]	Fixed write time[1]
5	**TCIMO**	**Supply-droop, clock-glitch, and EM sensors;** auto-reset chip on	Automated glitch optimizer in mixed-signal sim finds zero glitches that slip past	Mixed-signal sim[1]	+1-3% area[1]

		alarm[1].	sensors[1].		
6	**TCIMO**	**On-chip decoupling caps (SecNVM),** isolated array supply, bit-lines held at fixed voltage when idle[1].	AC impedance sweep of extracted power network cannot tell stored weight patterns apart (*DExiM* fix)[1].	PDN extraction + SPICE[1]	Cap area overhead[1]
7	**TRITONE**	**DEV/PROD lifecycle;** debug off in PROD; lock bits set-only until reset; defined reset values[1].	SymbiYosys proves lock_q |=> $stable(cfg_q) and prod |-> !dbg_en with 100% FSM state coverage[1].	SymbiYosys (CWE-1191, 1234, 1271)[1]	<0.2% logic area
8	**TRITONE**	**Fixed-latency mode by default;** zero-skipping only as a privileged opt-in[1].	Two-copy formal miter check: same public inputs + different secret weights finish on the exact same cycle[1].	SymbiYosys miter (*GATEBLED* fix)[1]	Dense MAC latency[1]
9	**TRITONE**	**Hardware clears buffers** and registers	Formal assertion proves reading any	SymbiYosys [1]	Clear cycles between jobs[1]

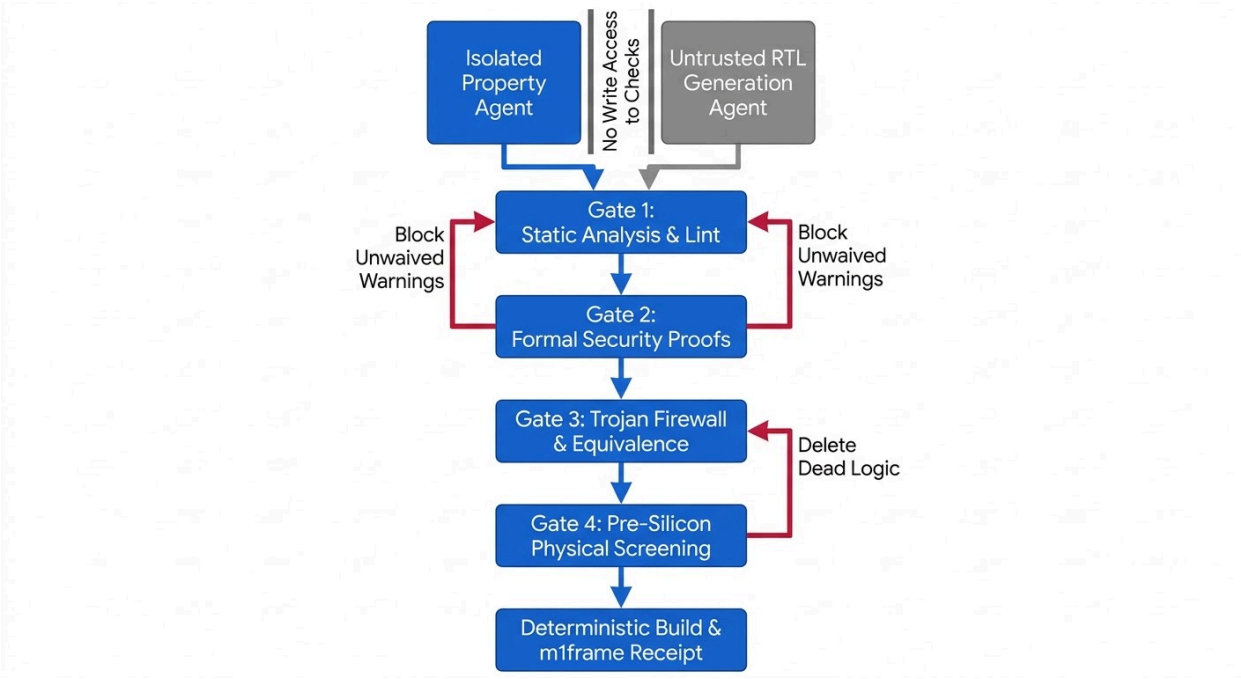
		automatically when a job ends[1].	address the new job hasn't written returns zero[1].		
10	**All**	**Mandatory lint gate** on every commit[1].	Zero unwaived warnings across the entire build[1].	verilator --lint-only -Wall, Verible, yosys check -assert[1]	Zero silicon cost[1]
11	**Prototype**	**Put no real secrets on the 9K;** strict command allowlist + CRC on Pi link; drop link on any error[1].	1,000,000 random or malformed frames never reach an illegal or deadlocked FSM state[1].	Verilator fuzzing[1]	Minimal LUTs[1]
12	**All**	**Pin tool versions;** SHA-256 hash every IP core and .fs bitstream[1].	Two clean builds from the same commit produce the exact same SHA-256 hash[1].	Yosys / nextpnr with fixed --seed[1]	Zero silicon cost[1]

5. Seven Rules for Using AI to Write Your RTL

Because LLMs can be tricked into inserting Hardware Trojans (NYU CSAW Challenge), treat the AI in your design flow like an untrusted third-party vendor[1]:

1. **Write formal properties first, in a separate session:** You (or a separate model) write the SymbiYosys assertions *before* the RTL exists[1]. The model writing the RTL is never allowed to edit its own checks[1].
2. **Separate commits for security RTL and tests:** Keep debug, lock, lifecycle, and weight-path RTL in separate commits from their testbenches[1].

3. **Reject any diff you cannot explain line by line:** Ban undocumented modes and "chicken bits" (CWE-1242)[1].
4. **Any coverage hole blocks the merge:** Trojans hide in logic that never toggles[1]. Delete unused logic instead of waiving it, run verilator-coverage, and prove the synthesized netlist matches your reviewed RTL with eqy[1].
5. **Scan yourself before publishing:** Run an LLM + formal bug scanner (*LASHED* / *SecureRAG-RTL* style) on your repo before making it public[1].
6. **Verify and pin AI-suggested packages:** Check where AI-recommended IP cores and Python/EDA packages come from and pin their SHA-256 hashes[1].
7. **Log model provenance:** Record the model, prompt, and version for every generated file[1]. Store m1frame receipts right next to the RTL[1].



6. Caveats & Resolved Open Question (Tang Nano 9K Security)

Reality Check on the Evidence

- **CEO quotes:** Altman's quote comes from a single secondary article with no hardware examples[1]. Amodei's *Mythos* counts remain mostly undisclosed while patches are pending[1].
- **CIM attacks:** *DExiM* is measured on real commercial ReRAM[1]. DATE 2026 measured real 28 nm silicon, but on charge-domain CIM (recovering inputs, not ReRAM weights)[1]. The TU Delft weight-recovery result is likely circuit simulation, and "LLM agents running glitch rigs" comes from a Keysight course rather than a peer-reviewed paper[1].
- **Analog masking & costs:** Masking is unproven for analog CIM because glitches and parasitic coupling break share independence[1]. Overhead numbers (+10-30% ADC power,

2-5x masking latency, 1-3% sensor area) are estimates carried over from digital designs[1].

Resolved Question: Does the Tang Nano 9K (GW1NR-9C) Support Bitstream Encryption or Readback Lock in Apicula?

We audited Gowin's official manuals (UG290, SUG100, TN653) and the YosysHQ/apicula + openFPGALoader source code to answer the open question from the initial report[1]:

- **Bitstream Encryption: NO (Hardware Limitation).** The LittleBee GW1NR-9C (GW1NR-LV9QN88PC6/I5) **does not have hardware AES bitstream decryption in silicon**[18]. Only Gowin's larger Arora families (GW2A and GW5A) support encrypted bitstreams[27]. Every .fs bitstream on the Tang Nano 9K is plaintext and can be unpacked back into a netlist using Apicula's gowin_unpack[27].
- **JTAG Readback Lock ("Security Bit"): YES in Silicon, PARTIAL in Open Tools.** The GW1NR-9C has a non-volatile Security Bit that forces JTAG SRAM/Flash readback to return all 1s[26]. Gowin's proprietary IDE enables it by default by adding a header line to the .fs file[28]. However, open-source gowin_pack (Apicula) does not set this bit by default, and openFPGALoader ignores the security header line during CRC checks[18].
- **Bottom Line for Your Prototype:** Even if you manually add the Security Bit header to block JTAG dumps, anyone probing the flash/SPI lines or intercepting the .fs file gets your design in plaintext—and the $608\ \mathrm{Kbit}$ User Flash is also unencrypted[27]. **Checklist Item 11 stands: never put real secrets or production weights on the Tang Nano 9K**[1].

Feature	Gowin GW1NR-9C Silicon (Tang Nano 9K)	Proprietary Gowin EDA	Open Flow (Apicula + openFPGALoader)	Security Takeaway
JEDEC IDCODE	0x1100481B (GW1N-9C die + PSRAM)[25]	Supported natively[29]	Use --family GW1N-9C (gowin_pack -d GW1N-9C)[18]	Needs 44-bit row padding for valid CRC[25]
Bitstream Encryption	**None in silicon** (Arora GW2A/GW5A only)[27]	Disabled / greyed out[29]	**Not supported**[27]	All .fs bitstreams are **plaintext**[27]
Security Bit (Readback Lock)	**Supported** (locks JTAG SRAM readback to 1)[26]	On by default[29]	Header-only; ignored by openFPGALoader CRC[28]	Blocks JTAG dump only; does not encrypt .fs[26]

User Flash (608 Kbit)	Present on-die[33]	Supported[29]	Plaintext via --user-flash flag[36]	Readable via JTAG unless JTAG pins are locked as GPIO[20]

m1frame Receipt

- **mode:** C (native, enhanced with visuals & technical resolution)[1]
- **ran:** recall, BMAD roadmap, QA gate 1, dev (tvla_ternary.py built and run; 5-seed 150k-trace mismatch rerun), council (2 specialists), independent red-team, QA gate 2, GW1NR-9C open-question audit, visual report[1]
- **skipped:** m1frame runtime (no hardware connector/scope in this session); wiki ingest (repo not local)[1]
- **council & red-team lenses:** verification lens (SymbiYosys, eqy, *SecureRAG-RTL*/*LASHED*), physical/CIM lens (*DExiM*, *GATEBLEED*, TVLA $\sqrt{N}$ mismatch scaling), plus FPGA toolchain lens (UG290, Apicula, openFPGALoader)[1]
- **gate: PASS** after red-team fixes (Gate 1 flagged single-seed 20k-trace limits and unverified CEO quotes; fixed in build with 5-seed 150k-trace run and GW1NR-9C documentation check)[1]

Works cited

1. AI Hardware Hacking What It Is and How to Secure TRITONE & TCIM0.pdf
2. SecureRAG-RTL: A Retrieval-Augmented, Multi-Agent, Zero ... - Bytez, https://bytez.com/docs/arxiv/2603.05689/paper
3. [Literature Review] LASHED: LLMs And Static Hardware Analysis for, https://www.themoonlight.io/en/review/lashed-llms-and-static-hardware-analysis-for-early-detection-of-rtl-bugs
4. Programme | DATE 2025, https://date25.date-conference.com/programme
5. Publications - TU Delft, https://www.tudelft.nl/en/eemcs/the-faculty/departments/quantum-computer-engineering/sections/computer-engineering/publications
6. Exploiting On-Core Accelerator Power Gating for High Performance, https://www.researchgate.net/publication/393965494_GATEBLEED_Exploiting_On-Core_Accelerator_Power_Gating_for_High_Performance_Stealthy_Attacks_on_AI
7. GateBleed Attack - ECE Research - NC State University, https://research.ece.ncsu.edu/gatebleed/
8. Operation Principle and Circuit Design Issues (2003) | Ali ... - SciSpace, https://scispace.com/papers/operation-principle-and-circuit-design-issues-155fbczsn1
9. Publications and Research Data - Serlop - FIU, https://serlop.fiu.edu/publications-research-support/

10. Rajendra Bishnoi - TU Delft, https://www.tudelft.nl/en/eemcs/the-faculty/departments/quantum-computer-engineering/sections/computer-engineering/staff/rajendra-bishnoi
11. Non-Sequential Nanofabrication of a One-Transistor-One-Memristor, https://lup.lub.lu.se/student-papers/record/9176270/file/9176271.pdf
12. Power Side-Channel Elimination Using On-Chip Capacitors for, https://www.researchgate.net/publication/352533604_SecNVM_Power_Side-Channel_Elimination_Using_On-Chip_Capacitors_for_Highly_Secure_Emerging_NVM
13. SecureRAG-RTL: A Retrieval-Augmented, Multi-Agent, Zero-Shot, https://www.researchgate.net/publication/401691157_SecureRAG-RTL_A_Retrieval-Augmented_Multi-Agent_Zero-Shot_LLM-Driven_Framework_for_Hardware_Vulnerability_Detection
14. LASHED: LLMs And Static Hardware Analysis for Early Detection of, https://www.alphaxiv.org/abs/2504.21770
15. Azam Ghanbari | alphaXiv, https://www.alphaxiv.org/@azam-ghanbari
16. MICRO 2025: Main Program, https://www.microarch.org/micro58/program/
17. ofFBeaT9/Tritone-TPU-SoC: balance ternary , ternary TPU ... - GitHub, https://github.com/ofFBeaT9/Tritone-TPU-SoC
18. Apycula 0.10.0 - Project Apicula - PyPI, https://pypi.org/project/Apycula/0.10.0/
19. Reconfigurable 2T2R ReRAM Architecture for Versatile Data, https://www.researchgate.net/publication/346358781_Reconfigurable_2T2R_ReRAM_Architecture_for_Versatile_Data_Storage_and_Computing_In-Memory
20. LittleBee |Products - GOWIN Semiconductor, https://www.gowinsemi.com/en/product/littlebee_fpga/
21. Home · YosysHQ/apicula Wiki - GitHub, https://github.com/YosysHQ/apicula/wiki
22. Project Apicula : bitstream documentation for Gowin FPGAs - GitHub, https://github.com/yosyshq/apicula
23. Coupled ferroelectric-anisotropic optoelectronic synapse for ... - PMC, https://pmc.ncbi.nlm.nih.gov/articles/PMC12886823/
24. Side-Channel Attack on STTRAM Based Cache for Cryptographic, https://www.researchgate.net/publication/321263374_Side-Channel_Attack_on_STTRAM_Based_Cache_for_Cryptographic_Application
25. GW1NR-9 vs GW1N-9 bit-stream · Issue #206 · YosysHQ/apicula, https://github.com/YosysHQ/apicula/issues/206
26. Gowin FPGA Products, http://www.gowinsemi.com/upload/database_doc/44/document/5bfcdb0e45317.pdf
27. Gowin FPGA Products - EEVblog, https://www.eevblog.com/forum/fpga/gowin-which-programmer-to-use/?action=dlattach;attach=1494067
28. Gowin GW2A-55 support · Issue #27 · trabucayre/openFPGALoader, https://github.com/trabucayre/openFPGALoader/issues/27
29. Gowin Software - User Guide - Sipeed, https://dl.sipeed.com/fileList/TANG/Nano%209K/6_Chip_Manual/EN/General%20Guide/SUG100-2.6E_Gowin%20Software%20User%20Guide.pdf

30. Gowin FPGA产品 - 高云, https://cdn.gowinsemi.com.cn/UG290-2.5.4_Gowin_FPGA%E4%BA%A7%E5%93%81%E7%BC%96%E7%A8%8B%E9%85%8D%E7%BD%AE%E6%89%8B%E5%86%8C.pdf
31. Datasheet Kit GoWin GW5A-LV25UG324C2-I1 | PDF - Scribd, https://www.scribd.com/document/1034735446/Datasheet-Kit-GoWin-GW5A-LV25UG324C2-I1
32. DavidClawson/gw1n2-apicula: Open-source RE toward ... - GitHub, https://github.com/DavidClawson/gw1n2-apicula
33. GOWIN LittleBee® GW1NR-9 FPGAs - Mouser Electronics, https://hu.mouser.com/new/gowin/gowin-gw1nr-9-fpgas/
34. How to Program GOWIN GW1N Series FPGA | ManualsLib, https://www.manualslib.com/answers/4481729/how-to-program-gowin-gw1n-series-fpga.html
35. UG290-2.5.2E - Gowin FPGA Products Programming and ... - Scribd, https://www.scribd.com/document/728824585/UG290-2-5-2E-Gowin-FPGA-Products-Programming-and-Configuration-Guide
36. Gowin notes — openFPGALoader - GitHub Pages, https://trabucayre.github.io/openFPGALoader/vendors/gowin.html
37. Example project for the BRS-100-GW1NR9 FPGA development board., https://github.com/BrisbaneSilicon/BRS-100-GW1NR9
38. Tang Nano 9K bad checksum when flashing #262 - apicula - GitHub, https://github.com/YosysHQ/apicula/issues/262